

Design Pattern Report: Observer Pattern

Group 52

Nguyễn Minh Huy (Student ID: 25125083)

July 10, 2026

Contents

1 Real-world Problem

Consider a modern video streaming application like YouTube. In this application, creators upload videos to channels, and subscribers wish to receive updates whenever a new video is published. The system needs to accommodate various types of notifications:

- Email notifications sent to desktop users.
- Push notifications delivered to mobile applications.
- Broadcast updates displayed on Smart TVs.

A user must be able to subscribe to or unsubscribe from a channel at any time. The core challenge is establishing a system where a YouTube channel can broadcast updates to an arbitrary and dynamic set of subscribers without knowing their concrete classes or notification mechanisms.

2 Naive Solution (Without Design Pattern)

In a naive implementation, the `YouTubeChannel` class holds lists of concrete subscriber objects directly. Below is the C++ representation of this approach:

```
1 // Refer to Observer/src/naive.cpp for full implementation details
2 class NaiveYouTubeChannel {
3 private:
4     std::string name;
5     std::vector<EmailSubscriber*> emailSubscribers;
6     std::vector<MobileSubscriber*> mobileSubscribers;
7
8 public:
9     void subscribeEmail(EmailSubscriber* sub) {
10         emailSubscribers.push_back(sub);
11     }
12     void subscribeMobile(MobileSubscriber* sub) {
13         mobileSubscribers.push_back(sub);
14     }
15
16     void uploadVideo(const std::string& videoTitle) {
17         // Direct method invocation on concrete objects
18         for (auto sub : emailSubscribers) {
19             sub->receiveEmailNotification(name, videoTitle);
20         }
21         for (auto sub : mobileSubscribers) {
22             sub->receivePushNotification(name, videoTitle);
23         }
24     }
25 };
```

Listing 1: Naive Implementation of YouTube Channel

3 Problems with the Naive Solution

The naive approach exhibits several major architectural flaws:

1. **Tight Coupling:** The `NaiveYouTubeChannel` is tightly coupled to concrete classes like `EmailSubscriber` and `MobileSubscriber`. It must know their names and exactly which method to invoke (e.g., `receiveEmailNotification` vs. `receivePushNotification`).
2. **Violation of the Open-Closed Principle (OCP):** If we want to introduce a new type of subscriber (such as `TVSubscriber` or `SMSSubscriber`), we must modify the `NaiveYouTubeChannel` class. This requires adding a new list, a new subscribe method, and altering the notification loop inside `uploadVideo`.
3. **Rigidity:** Code becomes difficult to maintain as the number of subscriber types grows, leading to bloated classes and repeated logic.

4 Introduction to the Observer Pattern

The **Observer Pattern** is a behavioral design pattern that defines a one-to-many dependency between objects. When one object (the **Subject** or **Publisher**) changes state, all its dependents (the **Observers** or **Subscribers**) are notified and updated automatically.

Key concepts:

- **Subject:** Knows its observers and provides an interface to attach or detach them.
- **Observer:** Defines an updating interface for objects that should be notified.
- **ConcreteSubject:** Stores state of interest to ConcreteObservers and sends notifications.
- **ConcreteObserver:** Implements the Observer updating interface to keep its state consistent with the subject.

5 General Class Diagram

In the general Observer pattern structure:

- A `Subject` maintains a list of references to `Observer` objects.
- The `Subject` exposes interfaces: `attach(Observer)`, `detach(Observer)`, and `notify()`.
- `Observer` exposes the `update()` method interface.

[Insert General Observer UML Diagram Here]

Figure 1: General Observer Design Pattern UML Diagram

6 Class Diagram for the YouTube Problem

Applying the pattern to our problem:

- `YouTubeChannelInterface` acts as the Subject interface.
- `YouTubeChannel` acts as the ConcreteSubject.
- `Subscriber` acts as the Observer interface.
- `EmailSubscriberPattern`, `MobileSubscriberPattern`, and `TVSubscriberPattern` act as the ConcreteObservers.

[Insert YouTube Observer UML Diagram Here]

Figure 2: UML Class Diagram for YouTube Subscription System

7 Implementation of the Observer Pattern

By decoupling the classes through abstract interfaces, our refactored implementation allows adding new subscriber types at runtime without modifying the subject.

```
1 // Refer to Observer/src/pattern.cpp for full implementation details
2 class Subscriber {
3 public:
4     virtual ~Subscriber() = default;
5     virtual void update(const std::string& channelName, const std::
        string& videoTitle) = 0;
6 };
7
8 class YouTubeChannel : public YouTubeChannelInterface {
9 private:
10     std::string name;
11     std::vector<Subscriber*> subscribers;
12
13 public:
14     void subscribe(Subscriber* s) override { subscribers.push_back(s);
15     }
16     void unsubscribe(Subscriber* s) override { /* Remove from list */ }
17     void notify(const std::string& videoTitle) override {
18         for (auto s : subscribers) {
19             s->update(name, videoTitle);
20         }
21     }
22 };
```

Listing 2: Refactored Observer Implementation

8 Pros and Cons

8.1 Pros

- **Loose Coupling:** The Subject and Observers are loosely coupled. The Subject only knows that the observer implements a specific interface.
- **Support for Broadcast Communication:** Unlike direct calls, notifications are broadcast automatically to all registered subscribers.
- **Open-Closed Principle:** We can introduce new concrete observer classes without changing the subject's code.

8.2 Cons

- **Lapsed Listener Problem:** Observers must be carefully deregistered when destroyed, or they remain in memory and get called, causing undefined behavior or leaks.
- **Unintended Update Chains:** If observers trigger updates on subjects, it can lead to complex and hard-to-debug cascading updates or infinite loops.
- **Ordering Concerns:** There is no guarantee in which order the observers will be notified.

9 Other Real-world Applications

9.1 Web Development

- **Event Handling in JavaScript:** The DOM's `addEventListener` uses the Observer pattern to attach event handlers to HTML elements.
- **State Management (Redux/Vuex):** Views subscribe to changes in a global store and re-render automatically when state mutations occur.

9.2 Mobile Development

- **RxSwift / RxJava / RxKotlin:** Reactive streams rely heavily on observing data sequences and binding them directly to UI components.
- **SwiftUI/Android Jetpack Compose:** Declarative UI frameworks use property wrappers like `@Published` and state flows to push model updates directly to the view.

10 Quiz

Here are some interactive questions to review the Observer pattern:

1. What type of design pattern is the Observer Pattern?

- A) Creational
 - B) Structural
 - C) Behavioral
 - D) Architectural
2. Which SOLID principle does the Observer pattern primarily help satisfy when adding new subscribers?
- A) Liskov Substitution Principle
 - B) Single Responsibility Principle
 - C) Interface Segregation Principle
 - D) Open-Closed Principle
3. What is the “Lapsed Listener” problem in the Observer pattern?
- A) The subject stops listening to updates.
 - B) Observers are not notified because the subject crashed.
 - C) An observer is deleted without unsubscribing, causing memory leaks/crashes.
 - D) Multiple subjects occupy the same listener list.
4. Which of the following is a key advantage of the Observer pattern?
- A) It ensures observers are updated sequentially.
 - B) It decouples the subject from concrete observer details.
 - C) It reduces memory overhead of notifications.
 - D) It prevents subjects from changing their state.

Answer Key: 1-C, 2-D, 3-C, 4-B.

11 References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Refactoring.Guru. *Design Patterns: Observer*. <https://refactoring.guru/design-patterns/observer>